

INFORME TÉCNICO

GroupApp

Sistema de Mensajería Distribuida

Proyecto 1 — ST0263 Tópicos Especiales en Telemática / SI3007 Sistemas Distribuidos
2026-1

1. Resumen Ejecutivo

GroupApp es una aplicación de mensajería grupal basada en arquitectura de microservicios. El sistema está dividido en servicios de autenticación, gestión de grupos/canales, mensajería y notificaciones en tiempo real, con persistencia desacoplada por dominio y comunicación asíncrona por broker de mensajes.

La solución objetivo de despliegue contempla una instancia EC2 por microservicio (más componentes compartidos de infraestructura), y frontend estático desplegado en Amazon S3.

2. Arquitectura del Sistema

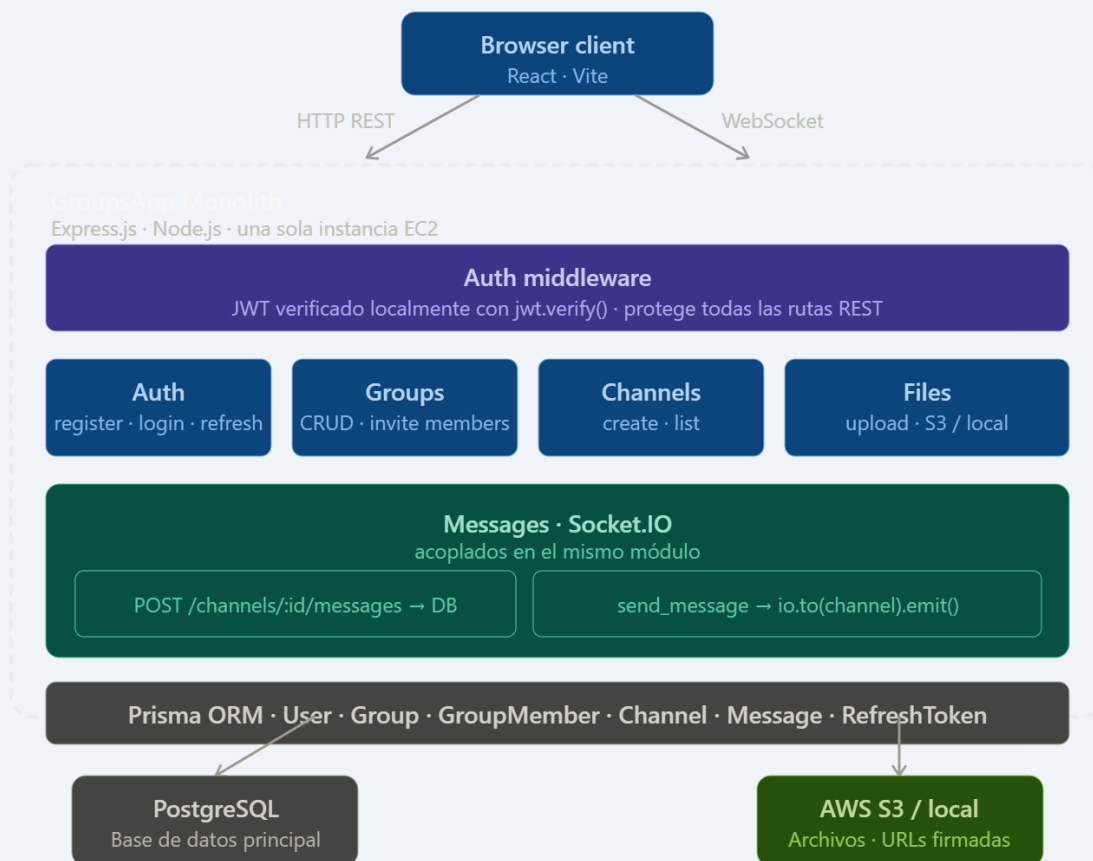
2.1 Componentes Principales

- Frontend (React + Vite): cliente web para autenticación, gestión de grupos/canales y chat.
- auth-service (Node.js + Express + Prisma): registro, login, refresh token.
- groups-service (Node.js + Express + Prisma): grupos, membresías, invitaciones y canales.
- messages-service (Node.js + Express + Prisma): historial de mensajes, envío, adjuntos.
- notifications-service (Node.js + Socket.IO + RabbitMQ): fan-out de eventos a clientes conectados.
- RabbitMQ: bus de eventos entre microservicios.
- PostgreSQL por servicio de dominio de datos (auth / groups / messages).
- Amazon S3: almacenamiento de archivos adjuntos de mensajes.

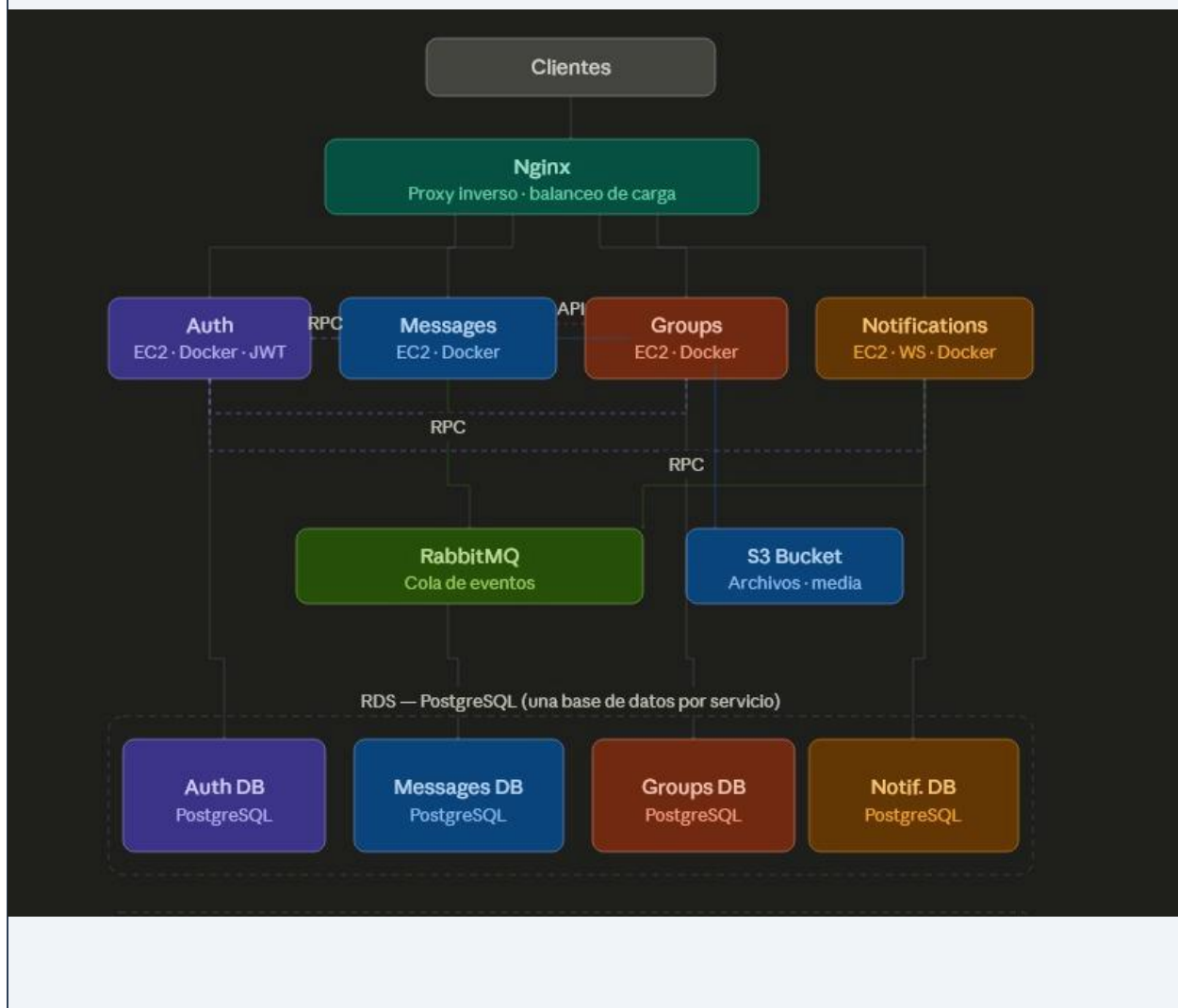
2.2 Diagrama de Arquitectura General

A continuación, se presenta el diagrama de alto nivel de la arquitectura del sistema:

[Diagrama de Arquitectura Monolítica — Vista General]



[Diagrama de Arquitectura Distribuida — Vista General]



2.3 Estilo Arquitectónico

La solución combina tres estilos de comunicación:

- REST síncrono para operaciones CRUD y consultas.
- Eventos asíncronos sobre RabbitMQ para desacoplar publicación y consumo.
- WebSocket (Socket.IO) para notificaciones en tiempo real al frontend.

2.4 Flujo de Alto Nivel

1. Usuario se autentica vía auth-service.
2. Frontend consume grupos/canales en groups-service.
3. Usuario envía mensaje al messages-service (texto y/o archivo).
4. messages-service persiste y publica evento message.sent.
5. notifications-service consume evento y emite receive_message por Socket.IO a la sala del canal.
6. Frontend refresca/consulta mensaje completo usando REST cuando aplica.

3. Tecnologías por Capa

Capa	Tecnologías
Frontend	React 18, Vite 5, React Router 6, Axios, Socket.IO Client
Backend	Node.js, Express, Prisma ORM, JWT, Multer
Mensajería	RabbitMQ (exchange tipo topic)
Base de datos	PostgreSQL (una instancia por dominio)
Archivos	AWS S3 (presigned / public URL)
Contenedores	Docker / Docker Compose (local)

4. APIs del Sistema

4.1 Auth Service — /auth

Base local (Docker Compose): <http://localhost:3001> → puerto interno 3000

Método	Endpoint	Descripción
POST	/auth/register	Crea usuario. Body: { username, email, password }
POST	/auth/login	Autentica usuario. Responde: accessToken, refreshToken, metadata.
POST	/auth/refresh	Renueva access token. Body: { refreshToken }

4.2 Groups Service

Base local (Docker Compose): <http://localhost:3002>

Método	Endpoint	Descripción
GET	/groups/my	Lista grupos del usuario autenticado.
POST	/groups	Crea grupo. Body: { name, description }
PUT	/groups/:id	Actualiza grupo (validando permisos).
POST	/groups/:id/invite	Invita usuario por email o userId.
POST	/groups/:groupId/channels	Crea canal dentro de un grupo. Body: { name }
GET	/groups/:groupId/channels	Lista canales del grupo.

4.3 Messages Service

Base local (Docker Compose): <http://localhost:3004>

Método	Endpoint	Descripción
GET	/messages/:messageId	Obtiene un mensaje por ID.
GET	/channels/:channelId/messages	Historial paginado. Query: page, limit.
POST	/channels/:channelId/messages	Envía mensaje (multipart/form-data). Campos: content, file. Persiste y emite evento message.sent.

4.4 Notifications Service

Base local (Docker Compose): <http://localhost:3003>

- REST: GET /health
- Socket.IO: handshake autenticado por JWT (socket.handshake.auth.token).
- Consume eventos message.sent de RabbitMQ y emite receive_message al canal con payload: { messageId, channelId, timestamp }.

5. Modelo de Datos

Cada microservicio de dominio gestiona su propia base de datos PostgreSQL. A continuación se describe el modelo Prisma por contexto.

[Diagrama de Modelo de datos]

AUTH DB — PostgreSQL

User		
int	id	PK
string	username	
string	email	
string	password	
datetime	createdAt	

tiene

RefreshToken		
string	id	PK
string	token	
int	userId	FK
datetime	createdAt	

GROUPS DB — PostgreSQL

Group		
int	id	PK
string	name	
string	description	
int	ownerId	
datetime	createdAt	

tiene

GroupMember		
int	id	PK
int	userId	
int	groupId	FK
string	role	

contiene

Channel		
int	id	PK
string	name	
int	groupId	FK
datetime	createdAt	

MESSAGES DB — PostgreSQL

Message		
int	id	PK
string	content	
string	fileUrl	
string	fileType	
string	status	
int	senderId	
int	channelId	
datetime	createdAt	

senderId → User.id (lógico)

channelId → Channel.id (lógico)

ownerId → User.id (lógico)

¿Por qué no FK cruzadas?

Referencias cross-domain son lógicas, no FK reales

5.1 Auth DB

User

Campo	Tipo	Notas
id	Int (PK, autoincrement)	
username	String	
email	String	Único
password	String	Hash bcrypt
createdAt	DateTime	

RefreshToken

Campo	Tipo	Notas
id	String (PK)	
token	String	Único
userId	Int (FK)	→ User.id
createdAt	DateTime	

5.2 Groups DB

Group / GroupMember / Channel

Entidad	Campo	Tipo	Notas
Group	id	String (PK)	
	name	String	
	description	String?	Nullable
	ownerId	Int	ID del propietario
	createdAt	DateTime	
GroupMember	id	String (PK)	
	userId	Int	
	groupId	String (FK)	→ Group.id
	role	Enum	ADMIN MEMBER
Channel	id	String (PK)	

Entidad	Campo	Tipo	Notas
	name	String	
	groupId	String (FK)	→ <i>Group.id</i>
	createdAt	DateTime	

5.3 Messages DB

Campo	Tipo	Notas
id	String (PK)	
content	String?	<i>Nullable</i>
fileUrl	String?	<i>Key o URL en S3</i>
fileType	String?	<i>Nullable</i>
status	Enum	<i>SENT DELIVERED READ</i>
senderId	Int	<i>ID lógico sin FK cruzada</i>
channelId	String	<i>ID lógico sin FK cruzada</i>
createdAt	DateTime	

6. Seguridad y Control de Acceso

- JWT Access Token para todos los endpoints protegidos.
- Refresh Token para renovación de sesión.
- Middleware de autenticación en cada servicio de dominio.
- Verificación de permisos en operaciones sensibles (updateGroup, invitaciones, etc.).
- Autenticación en handshake de WebSocket antes de aceptar la conexión.

Recomendaciones para producción:

- Rotación de secretos JWT mediante AWS Secrets Manager.
- TLS obligatorio (ALB + certificados ACM).
- CORS restringido por dominio (evitar * en producción).
- Políticas IAM mínimas para acceso a S3.

7. Plan de Pruebas

7.1 Pruebas Funcionales (Manuales)

7. Registro de usuario.
8. Login y obtención de tokens.
9. Refresh token.
10. Creación de grupo.
11. Listado de grupos del usuario (/groups/my).
12. Creación de canal.
13. Invitación de usuario.
14. Envío de mensaje de texto.
15. Envío de mensaje con archivo adjunto.
16. Consumo de notificación en tiempo real por Socket.IO.

7.2 Pruebas de Integración (Recomendadas)

- Auth + Groups: token válido accede a endpoints protegidos.
- Messages + RabbitMQ + Notifications: publicar message.sent y verificar emisión receive_message.
- Messages + S3: persistencia de objeto y recuperación de URL utilizable.

7.3 Pruebas No Funcionales (Recomendadas)

- Carga básica por endpoint crítico (k6 / Artillery).
- Disponibilidad de health checks en todos los servicios.
- Validación de reinicios de servicio ante caída controlada.

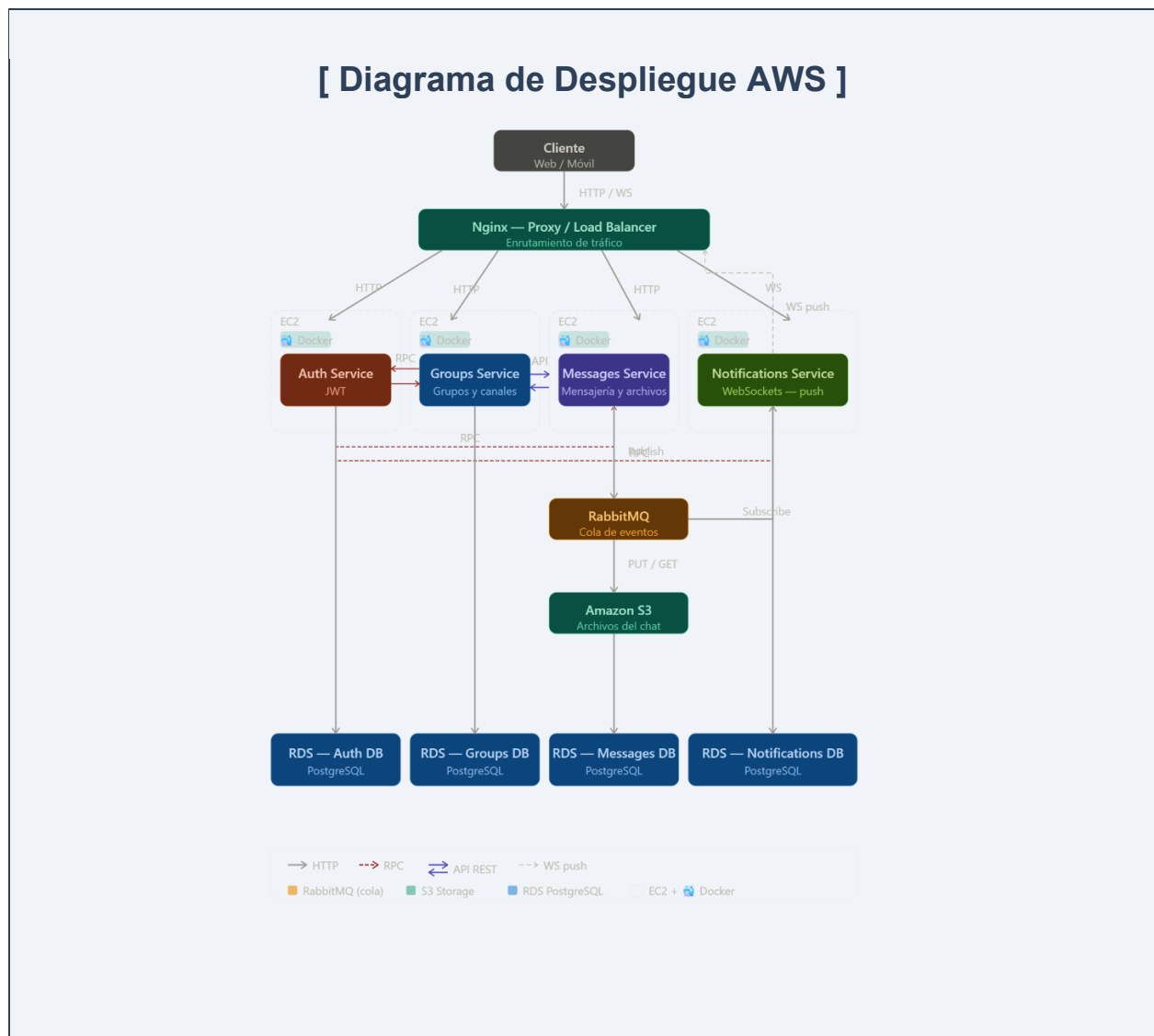
8. Despliegue en AWS

8.1 Topología Propuesta

Componente	Descripción
EC2-Auth	Instancia exclusiva para auth-service
EC2-Groups	Instancia exclusiva para groups-service
EC2-Messages	Instancia exclusiva para messages-service
EC2-Notifications	Instancia exclusiva para notifications-service

Componente	Descripción
EC2-RabbitMQ	Broker compartido (red privada)
Amazon RDS	3 instancias lógicas PostgreSQL (auth / groups / messages)
Amazon S3	Almacenamiento de archivos adjuntos
ALB	Application Load Balancer — routing por path (Nginx)
S3 (Frontend)	Hosting estático del cliente React + Vite
CloudFront	CDN + HTTPS para frontend (recomendado)

8.2 Diagrama de Despliegue AWS



8.3 Enrutamiento ALB

Path	Target Group
/auth/*	EC2-Auth
/groups/*	EC2-Groups
/channels/*, /messages/*	EC2-Messages
/socket.io/*	EC2-Notifications (WebSocket habilitado)

8.4 Pasos de Despliegue Backend (por microservicio)

17. Crear instancia EC2 (Amazon Linux 2023 o Ubuntu LTS).
18. Instalar runtime: Node.js LTS, PM2 / systemd.
19. Clonar repositorio o desplegar artefacto CI.
20. Configurar variables de entorno: PORT, DATABASE_URL, JWT_SECRET, RABBITMQ_URL, AWS_*.
21. Ejecutar migraciones Prisma (npx prisma migrate deploy).
22. Levantar servicio con PM2 o systemd.
23. Exponer puerto interno solo en Security Group privado; acceso externo únicamente por ALB.
24. Configurar health check: /health en cada servicio.

8.5 Frontend en S3

25. Build: npm ci && npm run build (genera carpeta dist/).
26. Subir dist/ al bucket S3 configurado para static website hosting.
27. Configurar política de bucket para lectura pública (o vía CloudFront OAC).
28. Apuntar variable VITE_API_URL al dominio del ALB.
29. (Recomendado) Activar CloudFront delante de S3 para HTTPS y caché global.

9. Operación y Observabilidad

- Logs centralizados en CloudWatch Logs por servicio.
- Métricas y alarmas: CPU / Memoria EC2, tasa de errores 5xx por target group, disponibilidad de RabbitMQ.

- Trazabilidad por request-id propagado entre servicios.
- Dashboards por servicio con alertas de degradación.

10. Riesgos Técnicos y Mejoras

Riesgos identificados

- 30. Ausencia de suite de tests automatizada.
- 31. Health checks no homogéneos entre servicios.
- 32. Valores de conexión hardcodeados en entorno local (host RabbitMQ en Compose) — deben parametrizarse completamente.
- 33. Falta de CI/CD formal para build, test y despliegue por servicio.

Mejoras prioritarias

- Implementar pruebas de integración y contract testing.
- Pipeline CI/CD por microservicio (GitHub Actions + despliegue automatizado).
- Centralizar secretos en AWS Secrets Manager / SSM Parameter Store.
- Añadir versionado de API y documentación OpenAPI por servicio.

11. Conclusión

GroupApp cumple una base sólida de sistema distribuido orientado a microservicios, con separación de dominios, comunicación asíncrona por eventos y canal de tiempo real para notificaciones. La estrategia de despliegue en AWS con EC2 por microservicio y frontend en S3 es viable y escalable para el contexto académico.

La solución se robustece al incorporar RDS administrado, ALB con routing por path, CloudFront, pruebas automatizadas y observabilidad centralizada mediante CloudWatch, cubriendo los criterios de evaluación de arquitectura escalable, datos distribuidos, comunicaciones remotas y aspectos adicionales del sistema distribuido definidos en la especificación del Proyecto 1.